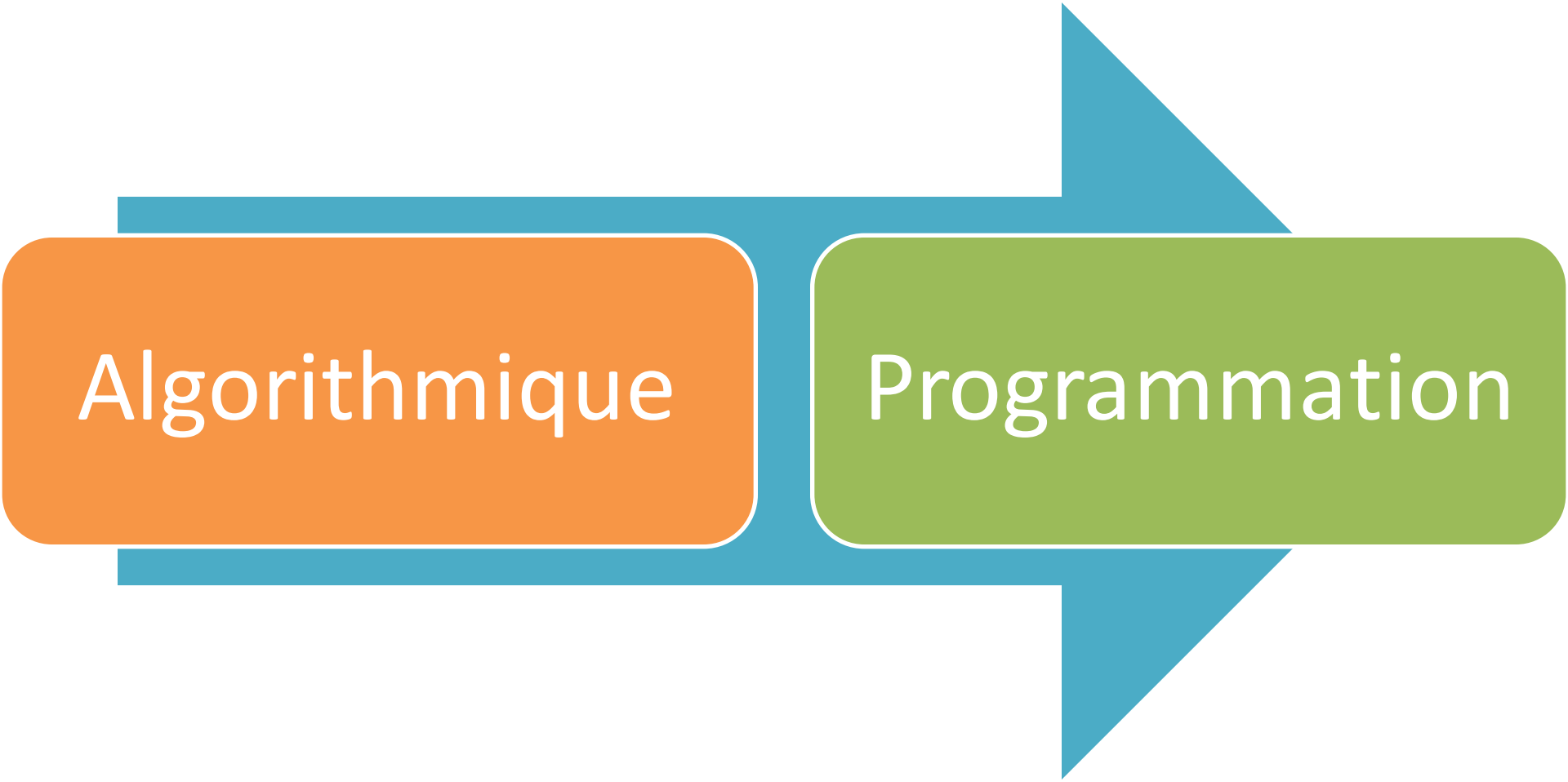
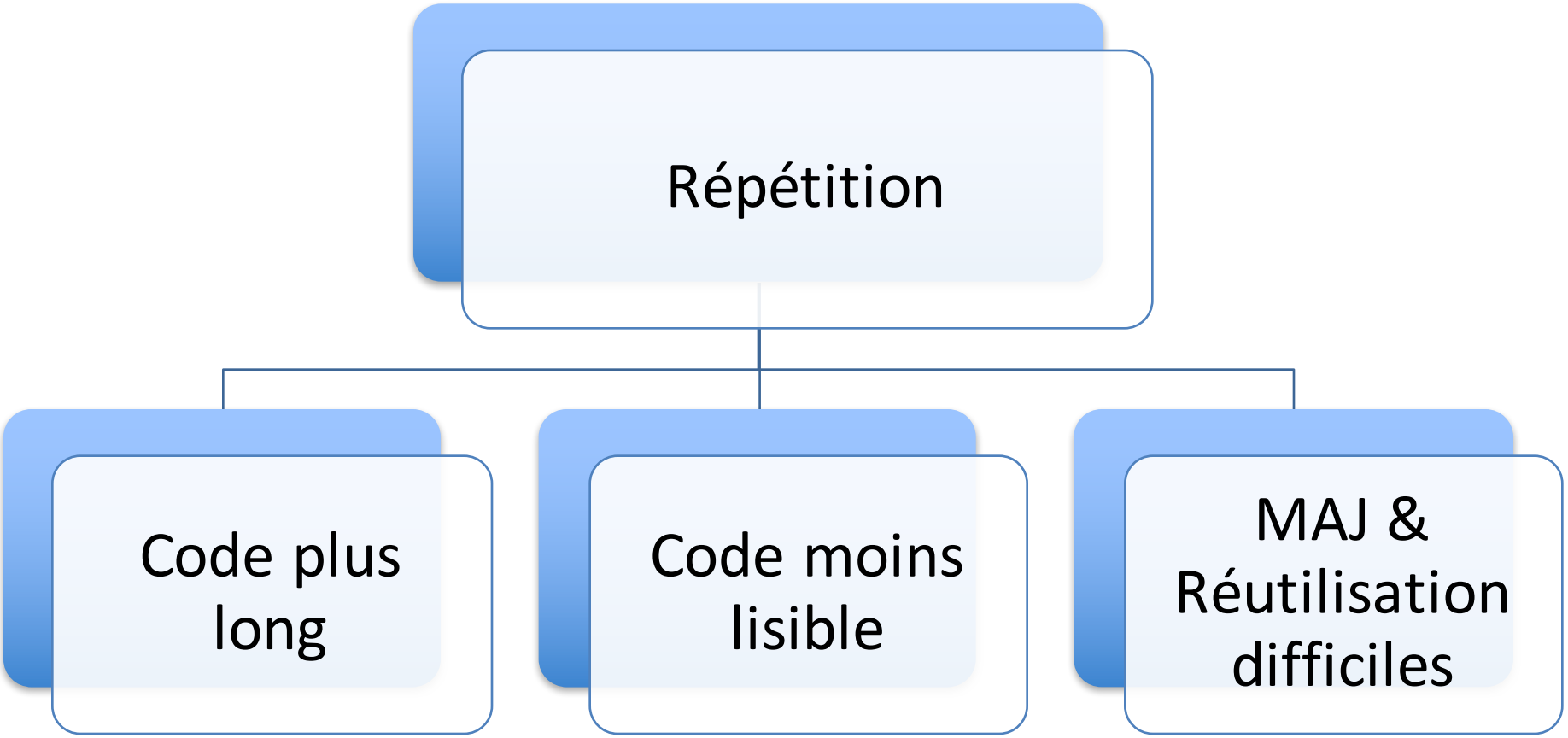


Chapitre 6: Les procédures et fonctions

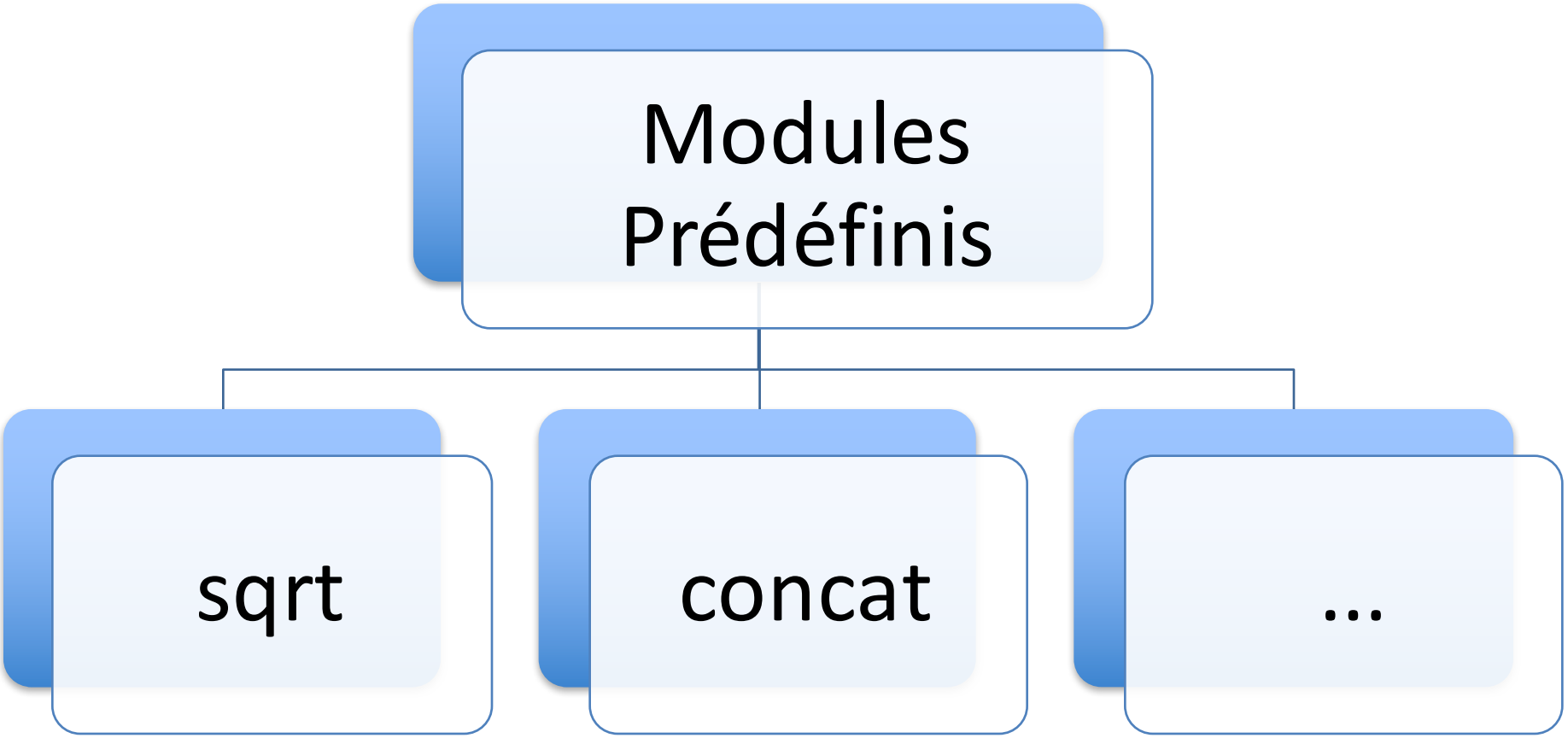
Algorithmes et Pascal

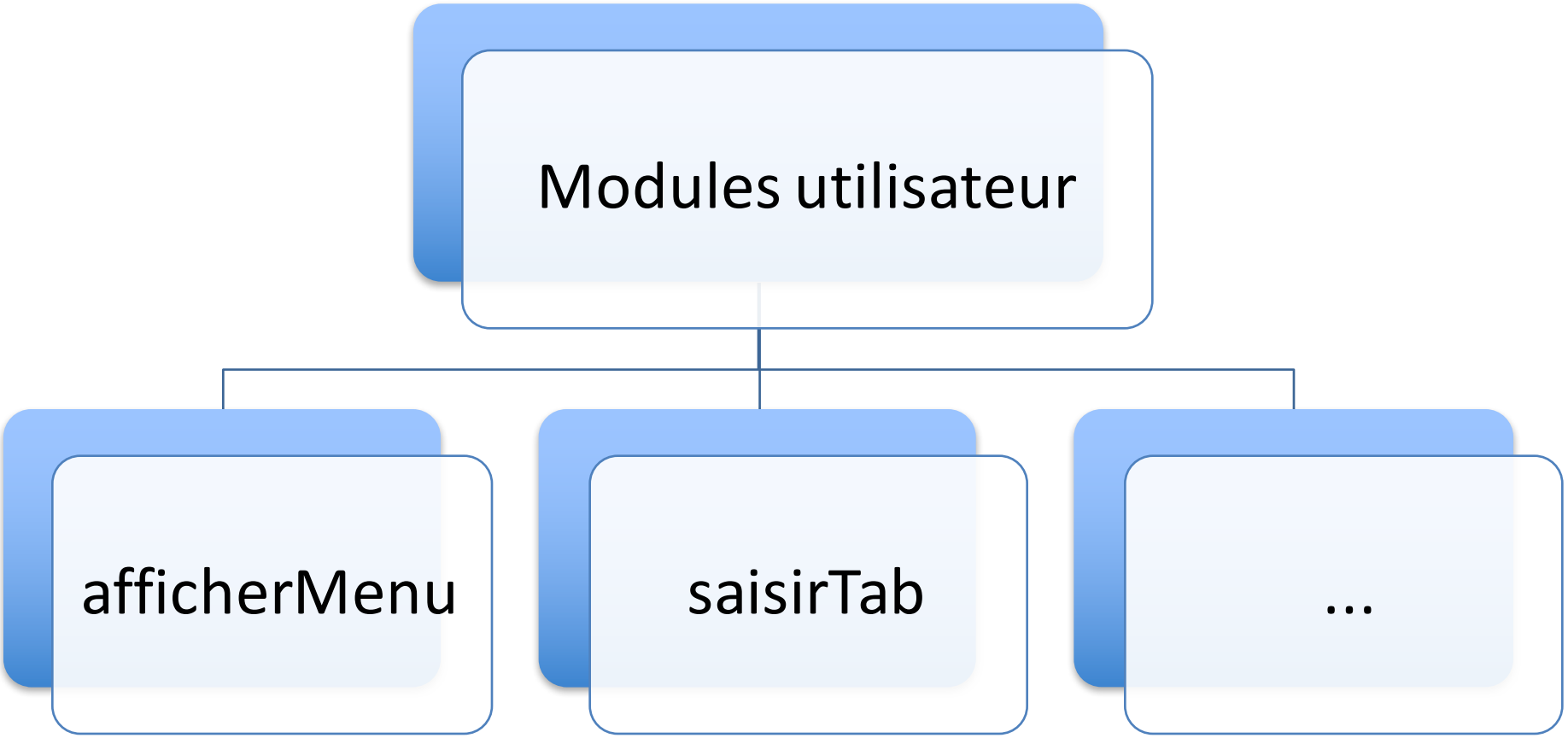
adrien.basse@uadb.edu.sn

Ecrire un programme qui permet de saisir trois tableaux de 10 entiers et qui les affiche.


Répétition
Code plus long
Code moins lisible
MAJ & Réutilisation difficiles

Un **module** ou sous-programme est un regroupement d'instructions disposant d'un nom et accessible autant de fois que nécessaire.


Modules
Prédéfinis
sqrt
concat
...


Modules utilisateur
afficherMenu
saisirTab
...

Définition de module

```
procedure afficherMenu;
 Begin
  writeln('Faites votre choix');
  writeln ('1: Boursier');
  writeln ('2: Non Boursier');
  writeln ('3: Quitter');
end;
```

Appel de module

```
Begin
    repeat
      afficherMenu;
      readln(choix);
     until choix<>3;
End.
```

Définition de module

```
procedure somme;
var a,b,c, som:integer;
Begin
    write('Donner a, b et c') ;
     readln(a,b,c);
     som:=a+b+c;
     write('la somme est
',som);
end;
```

Appel de module

```
Begin
     somme;
      somme;
end.
```

Définition de module

```
Type typeTab= array[1..10] of integer;
```

```
procedure afficherTab(t: typeTab);
Var i:integer;
begin
   for i:=1 to 7 do
     write(tab[i]) ;
end;
```

Appel de module

```
Var t1,t2:typeTab;

Begin
     saisirTab(t1);
     saisirTab(t2);
    afficherTab(t1);
     afficherTab(t2);
end.
```

Deux types de modules

- Les procédures
- Les fonctions

Appel de fonctions

- carre:=sqr(3);
- posi:=pos(‘ou’,’bonjour’);

sqr est une fonction qui prend en argument un réel, calcule et renvoie son carré.

pos est une fonction qui prend en arguments deux chaines, calcule et renvoie la position de la première chaine dans la seconde.

Appel de procédures

- delete(ch,3, 2);
- Insert('ch',ch, 5);

delete est une procédure qui prend en arguments une chaine, un nombre et une position pour supprimer de la chaine un nombre de caractères à partir d'une position et ne renvoie aucune donnée.

insert est une procédure qui prend en arguments deux chaines et une position, insère la première chaine dans la seconde à la position donnée et ne renvoie aucune donnée.

Une fonction renvoie une valeur, une procédure n'en renvoie aucune.

Fonction comme procédure peuvent disposer d’arguments.

Syntaxe de définition d'une procédure sans argument

```
Procedure nomProcedure;
 Begin
      {mettre ici le contenu}
 end;
```

```
Procedure bonjour;
 Begin
    write ('bonjour le monde');
 end;
```

```
Begin
   bonjour;
end.
```

```
var i:integer;
Begin
 for i:=1 to 10 do
  bonjour;
 end.
```

Avec variables locales

```
Procedure bonjour10;
  var i: integer;
 Begin
    for i:=1 to 10 do
    write ('bonjour le monde');
 end;
```

```
Begin
   bonjour10;
end.
```

Une variable locale est une variable définie à l'intérieur d'un bloc (sous-programme).

Une variable locale n’est visible que dans le sous programme où elle est déclarée.

Variable locale et variable globale

```
var i: integer;
Procedure valeurI;
  var i: integer;
 Begin
   i:=1;
   write (‘i locale vaut ’,i);
 end;
 Begin
   i:=10;
   valeurI;
   write (‘i globale vaut ’,i);
 end.
```

Une variable globale est une variable définie hors d'un bloc (sous-programme).

Une variable globale est visible dans tout le programme.

En cas homonymie *(variable globale et locale de même nom)*, la variable locale est visible.

Variable locale et variable globale

```
var i,j: integer;
Procedure valeurI;
  var i: integer;
 Begin
   i:=1;
   write ('i=',i,' et j=',j);
 end;
 Begin
   i:=10;
   j:=10;
   valeurI;
   write ('i=',i,' et j=',j);
 end.
```

Proposer un sous programme pour permuter deux entiers déclarés globalement.

Proposer un sous programme qui calcule le double d'un entier déclaré globalement. L'affichage se fait dans la fonction principale.

Syntaxe de définition d'une procédure avec argument

```
Procedure nomProcedure(vari1:type1,vari2: type2);
  Begin
        {mettre ici le contenu}
  end;
```

```
Procedure bonjourN (n:integer);
  var i: integer;
 Begin
    for i:=1 to n do
    write ('bonjour le monde');
 end;
```

```
Var a:integer;
Begin
  write ('Donner un entier');
  readln (a);
  bonjourN(a);
 end.
```

Le passage des arguments est par défaut par valeur.

```
Var n:integer;
Begin
   n:=10;
   bonjourN(n*2);
   bonjourN(5);
 end.
```

```
Var j: integer;
Procedure pasPlusUn(i:integer);
 Begin
   i:=i+1;
   write ('i vaut ',i);
 end;
 Begin
   j:=10;
   pasPlusUn(j);
   write ('j vaut ',j);
 end.
```

```
Var i: integer;
Procedure pasPlusUn(i:integer);
 Begin
   i:=i+1;
   write ('i vaut ',i);
 end;
 Begin
   i:=10;
   pasPlusUn(i);
   write ('i vaut ',i);
 end.
```

Le passage des arguments par adresse (référence, variable) se fait avec le mot var.

```
Var j: integer;
Procedure plusUn(var i:integer);
 Begin
   i:=i+1;
   write (‘i vaut ’,i);
 end;
 Begin
   j:=10;
   plusUn(j);
   write (‘j vaut ’,j);
 end.
```